DO NOW — RECALL

Recursion: write and debug algorithms

Name: _____ Class: _____ Date: _____

1. SumTo(n) returns 1 when n=1; otherwise it returns n+SumTo(n-1). For SumTo(4), identify the call that returns first and the call that returns last.

2. Why must a recursive function contain a stopping condition?

3. A loop repeatedly halves n starting at 40 while n > 1 using integer division. List the next three n values.

4. A function PowerTwo(n) returns 1 when n=0, otherwise 2*PowerTwo(n-1). What does PowerTwo(3) return?

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

An algorithm is recursive whenever it uses a loop.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

First: SumTo(1). Last: SumTo(4). The nested calls finish before their callers return.

2. Model answer

Without a reachable base case, calls continue and the call stack grows until failure.

3. Model answer

20, 10, 5.

4. Model answer

PowerTwo(0)=1, PowerTwo(1)=2, PowerTwo(2)=4 and PowerTwo(3)=8.

Vocabulary — know these terms

termination — Reaching a condition where the algorithm stops.

recursive step — Processing part of a problem before calling the function on a simpler input.

precondition — An assumption or input rule required for an algorithm.

stack overflow — A failure that can occur when too many active calls consume stack space.

integer division — Division discarding any fractional part of the result.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Extend CountDigits for n = 0 and negative integers without overflowing on the minimum C# int value.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Trace CountDigits(67): write the call sequence and final value.

Question 2

A pupil writes return CountDigits(n) + 1; for the recursive step. Correct the line and explain the error.

NEW LEARNING

Read, annotate and explain

Find the smallest problem

A positive single-digit number needs one digit. So n < 10 is our base case.

Make progress

For n ≥ 10, integer division by 10 makes the number shorter: 4827 → 482 → 48 → 4.

Count on the way back

4 returns 1. Then 48 returns 2; 482 returns 3; 4827 returns 4.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Model CountDigits(582). Why is integer division essential?

Calls

582 → 58 → 5. Each integer division by 10 discards the last digit.

Base case

5 is a single digit, so CountDigits(5) returns 1.

Returns

CountDigits(58)=2; CountDigits(582)=3. The original number has 3 digits.

WE DO — GUIDED PRACTICE

Try → Discuss → Improve